

Chapter 4: Arrays and Linked Structures

Fundamentals of Python: Data Structures, Second Edition

Learning Objectives (1 of 2)

- Create arrays
- Perform various operations on arrays
- Determine the running times and memory usage of array operations
- Describe how costs and benefits of array operations depend on how arrays are represented in computer memory
- Create linked structures using singly linked nodes
- Perform various operations on linked structures with singly linked nodes

Learning Objectives (2 of 2)

- Describe how costs and benefits of operations on linked structures depend on how they are represented in computer memory
- Compare the trade-offs in running time and memory usage of arrays and linked structures

The Array Data Structure (1 of 6)

- Array – represents a sequence of items that can be accessed or replaced at given index positions:
 - Data structure underlying a Python list is an array
 - It is the primary implementing structure in the collections of Python
- A programmer can
 - Access and replace an array's items at given positions
 - Examine an array's length
 - Obtain its string representation
- A programmer cannot
 - Add or remove positions
 - Make the length of the array larger or smaller (length of an array is fixed when created)

The Array Data Structure (2 of 6)

- Python's **array** model includes an **array** class:
 - Behaves more like a list but is limited to storing numbers
- You will define a new class named **Array** that adheres to the restrictions mentioned earlier but can hold items of any type
- This **Array** class uses a Python list to hold its items
- The class defines methods that allow clients to use
 - The subscript operator `[]`
 - The **len** function
 - The **str** function
 - The **for** loop with array objects

The Array Data Structure (3 of 6)

- Table 4.1 Array Operations and the Methods of the **Array** Class

User's Array Operation	Method in the Array Class
<code>a = Array(10)</code>	<code>__init__(capacity, fillValue = None)</code>
<code>len(a)</code>	<code>__len__()</code>
<code>str(a)</code>	<code>__str__()</code>
<code>for item in a:</code>	<code>__iter__()</code>
<code>a[index]</code>	<code>__getitem__(index)</code>
<code>A[index] = newItem</code>	<code>__setitem__(index, newItem)</code>

The Array Data Structure (4 of 6)

- Code for the Array class (in arrays.py):

```
"""
File: arrays.py
An Array is like a list, but the client can use
only [], len, iter, and str.

To instantiate, use
<variable> = Array(<capacity>, <optional fill value>)

The fill value is None by default.
"""

class Array(object):
    """Represents an array."""

    def __init__(self, capacity, fillValue = None):
        """Capacity is the static size of the array.
        fillValue is placed at each position."""
        self.items = list()
        for count in range(capacity):
            self.items.append(fillValue)
```

The Array Data Structure (5 of 6)

- Code for the Array class (in arrays.py) (continued):

```
def __len__(self):  
    """-> The capacity of the array."""  
    return len(self.items)  
  
def __str__(self):  
    """-> The string representation of the array."""  
    return str(self.items)  
  
def __iter__(self):  
    """Supports traversal with a for loop."""  
    return iter(self.items)  
  
def __getitem__(self, index):  
    """Subscript operator for access at index."""  
    return self.items[index]  
  
def __setitem__(self, index, newItem):  
    """Subscript operator for replacement at index."""  
    self.items[index] = newItem
```


The Array Data Structure (6 of 6)

- Shell session showing use of an array:

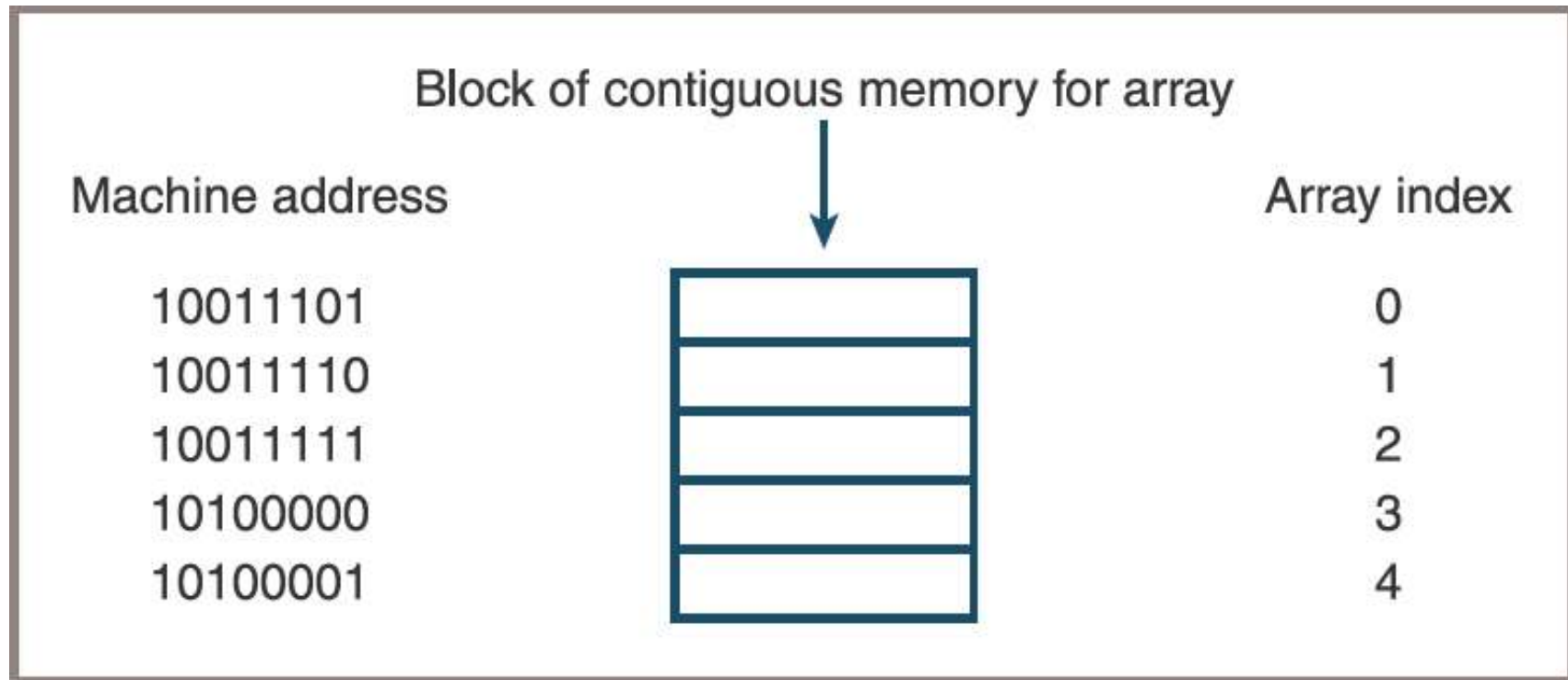
```
from arrays import Array
>>> a = Array(5)           # Create an array with 5 positions
>>> len(a)                 # Show the number of positions
5
>>> print(a)               # Show the contents
[None, None, None, None, None]
>>> for i in range(len(a)): # Replace contents with 1..5
    a[i] = i + 1
>>> a[0]                   # Access the first item
1
>>> for item in a:         # Traverse the array to print all
    print(item)
1
2
3
4
5
```

Random Access and Contiguous Memory (1 of 2)

- Array indexing is a *random access* operation:
 - The computer obtains the location of the i th item by performing a constant number of steps
 - No matter how large the array, it takes the same amount of time to access the first item as it does to access the last item
- The computer supports random access for arrays by allocating a block of contiguous memory cells for the array's items

Random Access and Contiguous Memory (2 of 2)

Figure 4-1: A block of contiguous memory



Static Memory and Dynamic Memory

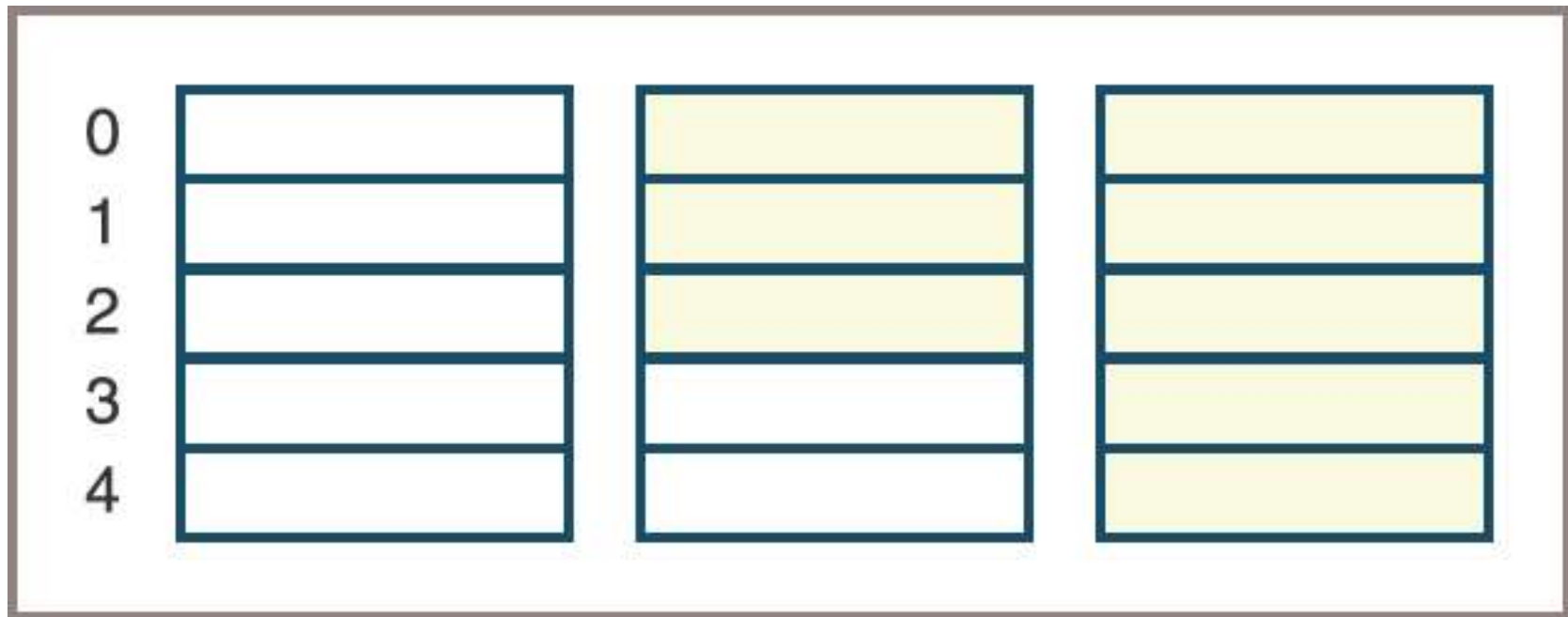
- Arrays in older languages were static data structures
- Modern languages allow dynamic arrays:
 - Occupies a contiguous block of memory and supports random access
- The Python **Array** class allows the programmer to specify the length of a dynamic array during instantiation
- Ways that programmers can read just an array's length
 - Create an array with a reasonable default size at program start-up
 - When the array cannot hold more data, create a new, larger array and transfer the data items from the old array
 - When the array seems to be wasting memory, decrease its length in a similar manner
- These adjustments happen automatically with a Python list

Physical Size and Logical Size (1 of 2)

- Physical size of an array
 - Its total number of array cells or the number used to specify its capacity when the array is created
- Logical size of an array
 - The number of items in it that should be currently available to the application
- Logical and physical size tell you several things:
 - If the logical size is 0, the array is empty
 - Otherwise, at any given time, the index of the last item in the array is the logical size minus 1
 - If the logical size equals the physical size, there is no more room for data in the array

Physical Size and Logical Size (2 of 2)

Figure 4-2: Arrays with different logical sizes



Operations on Arrays

- This section shows the implementation of several operations on arrays
- In these examples, assume the following data settings:

```
DEFAULT_CAPACITY = 5
```

```
logicalSize = 0
```

```
a = Array(DEFAULT_CAPACITY)
```

Increasing the Size of an Array

- The resizing process consists of three steps:
 - Create a new, larger array
 - Copy the data from the old array to the new array
 - Reset the old array variable to the new array object
- Code:

```
if logicalSize == len(a):  
    temp = Array(len(a) + 1)           # Create a new array  
    for i in range(logicalSize):      # Copy data from the old  
        temp[i] = a[i]                # array to the new array  
    a = temp                          # Reset the old array variable  
                                     # to the new array
```


Decreasing the Size of an Array

- The process of decreasing the size of an array is the inverse of increasing it
- Steps
 - Create a new, smaller array
 - Copy the data from the old array to the new array
 - Reset the old array variable to the new array object
- Code:

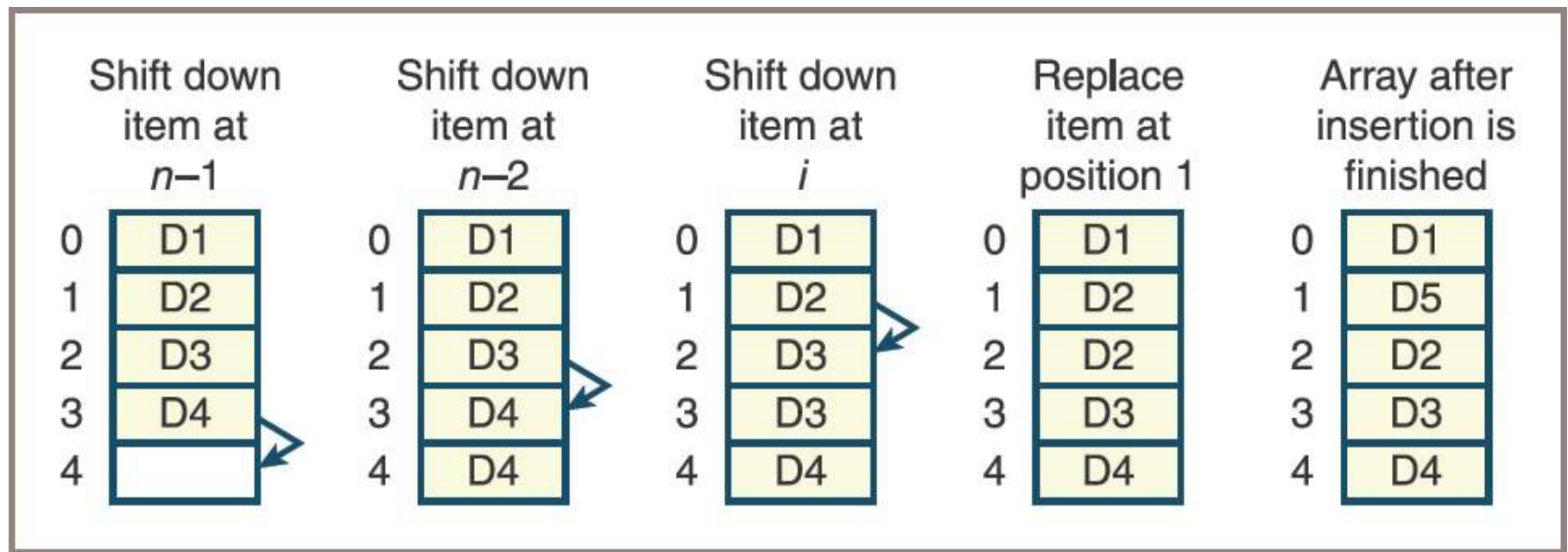
```
if logicalSize <= len(a) // 4 and len(a) >= DEFAULT_CAPACITY * 2:
    temp = Array(len(a) // 2)           # Create new array
    for i in range(logicalSize):        # Copy data from old array
        temp[i] = a[i]                 # to new array
    a = temp                            # Reset old array variable to
                                       # new array
```

Inserting an Item into an Array That Grows (1 of 3)

- In the case of an insertion, the programmer must do four things:
 - Check for available space before attempting an insertion and increase the physical size of the array, if necessary, as described earlier
 - Shift the items from the logical end of the array to the target index position down by one:
 - This process opens a hole for the new item at the target index
 - Assign the new item to the target index position
 - Increment the logical size by one

Inserting an Item into an Array That Grows (2 of 3)

Figure 4-3: Inserting an item into an array



Inserting an Item into an Array That Grows (3 of 3)

- Python code for the insertion operation:

```
# Increase physical size of array if necessary
# Shift items down by one position
for i in range(logicalSize, targetIndex, -1):
    a[i] = a[i - 1]
# Add new item and increment logical size
a[targetIndex] = newItem
logicalSize += 1
```

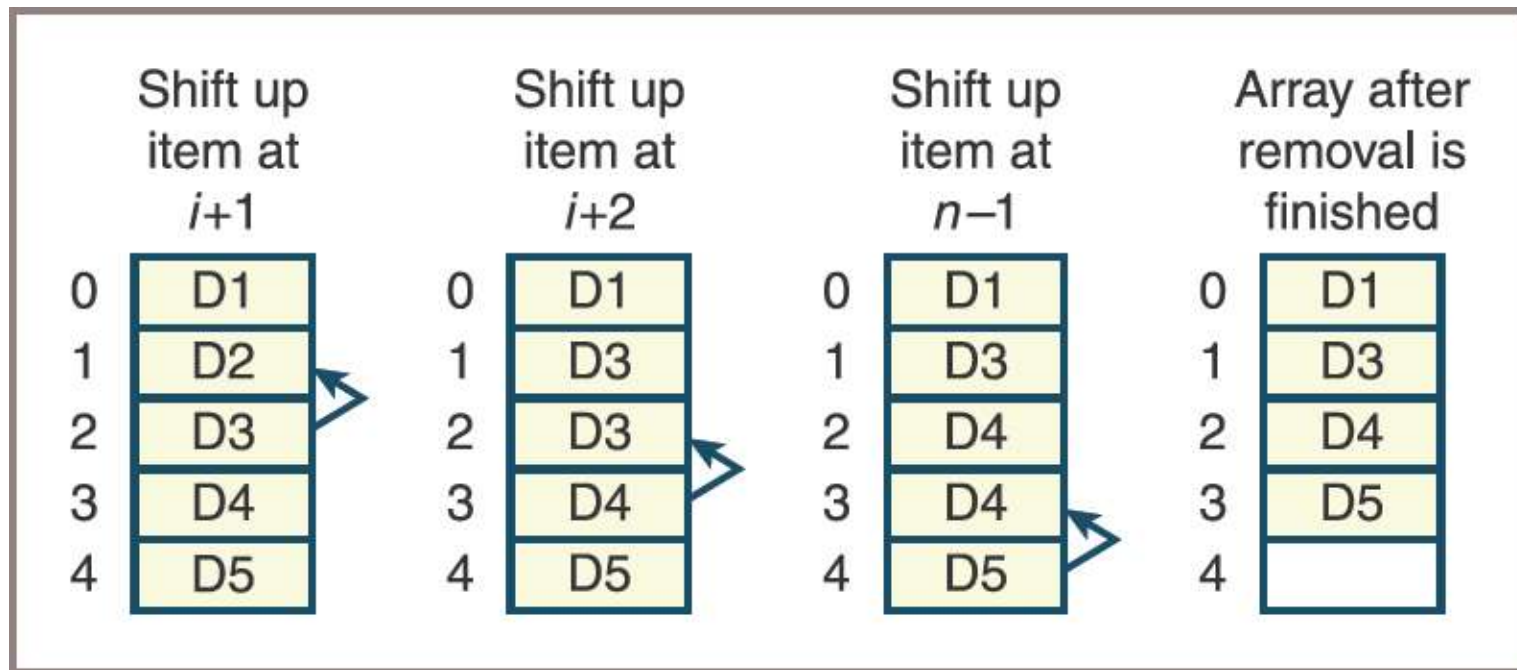
Removing an Item from an Array (1 of 2)

- Steps for removing an item from an array
 - Shift the items from the one following the target index position to the logical end of the array up by one. This process closes the hole left by the removed item at the target index
 - Decrement the logical size by one
 - Check for wasted space and decrease the physical size of the array, if necessary
- Code:

```
# Shift items up by one position
for i in range(targetIndex, logicalSize - 1):
    a[i] = a[i + 1]
# Decrement logical size
logicalSize -= 1
# Decrease size of array if necessary
```

Removing an Item from an Array (2 of 2)

Figure 4-4: Removing an item from an array



Complexity Trade-Off: Time, Space, and Arrays

- Table 4.2 The Running Times of Array Operations

Operation	Running Time
Access at i th position	$O(1)$, best and worse cases
Replacement at i th position	$O(1)$, best and worst cases
Insert at logical end	$O(1)$, average case
Insert at i th position	$O(n)$, average case
Remove from i th position	$O(n)$, average case
Increase capacity	$O(n)$, best and worst cases
Decrease capacity	$O(n)$, best and worst cases
Remove from logical end	$O(1)$, average case
Insert at i th position	$O(n)$, average case
Remove from i th position	$O(n)$, average case
Increase capacity	$O(n)$, best and worst cases
Decrease capacity	$O(n)$, best and worst cases

Two-Dimensional Arrays (Grids)

Figure 4-5: A two-dimensional array or grid with four rows and five columns

	Col 0	Col 1	Col 2	Col 3	Col 4
Row 0	0	1	2	3	4
Row 1	10	11	12	13	14
Row 2	20	21	22	23	24
Row 3	30	31	32	33	34

Processing a Grid

- A grid must recognize two methods that return the number of rows and the number of columns:
 - For purposes of discussion, these methods are named **getHeight** and **getWidth**
- Techniques for manipulating one-dimensional arrays are easily extended to grids
- The following code segment computes the sum of all the numbers in the variable grid:

```
sum = 0
for row in range (grid.getHeight()) :           # Go through rows
    for column in range (grid.getWidth()) :      # Go through columns
        sum += grid[row] [column]
```

Creating and Initializing a Grid (1 of 2)

- To create a **Grid** object,
 - Run the **Grid** constructor with three arguments: height, width, and an initial fill value
- Next session instantiates **Grid** with 4 rows, 5 columns, and a fill value of 0:

```
>>> from grid import Grid
>>> grid = Grid(4, 5, 0)
>>> print(grid)
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
```

Creating and Initializing a Grid (2 of 2)

- The following code segment traverses the grid to reset its cells to the values shown in Figure 4.5:

```
# Go through rows
for row in range(grid.getHeight()):
    # Go through columns
    for column in range(grid.getWidth()):
        grid[row][column] = int(str(row) + str(column))
```

Defining a Grid Class

- Code for the `Grid` class (in `grid.py`):

```
"""
Author: Ken Lambert
"""

from arrays import Array

class Grid(object):
    """Represents a two-dimensional array."""

    def __init__(self, rows, columns, fillValue = None):
        self.data = Array(rows)
        for row in range (rows):
            self.data[row] = Array(columns, fillValue)

    def getHeight(self):
        """Returns the number of rows."""
        return len(self.data)

    def getWidth(self):
        """Returns the number of columns."""
        return len(self.data[0])

    def __getitem__(self, index):
        """Supports two-dimensional indexing
        with [row][column]."""
        return self.data[index]

    def __str__(self):
        """Returns a string representation of the grid."""
        result = ""
        for row in range (self.getHeight()):
            for col in range (self.getWidth()):
                result += str(self.data[row][col]) + " "
            result += "\n"
        return result
```

Ragged Grids and Multidimensional Arrays

- Ragged grid
 - Has a fixed number of rows, but the number of columns of data in each row can vary
- An array of lists or arrays provides a suitable structure for implementing a ragged grid
- Dimensions can be added to the definition of a grid when necessary:
 - The only limit is the computer's memory

Linked Structures

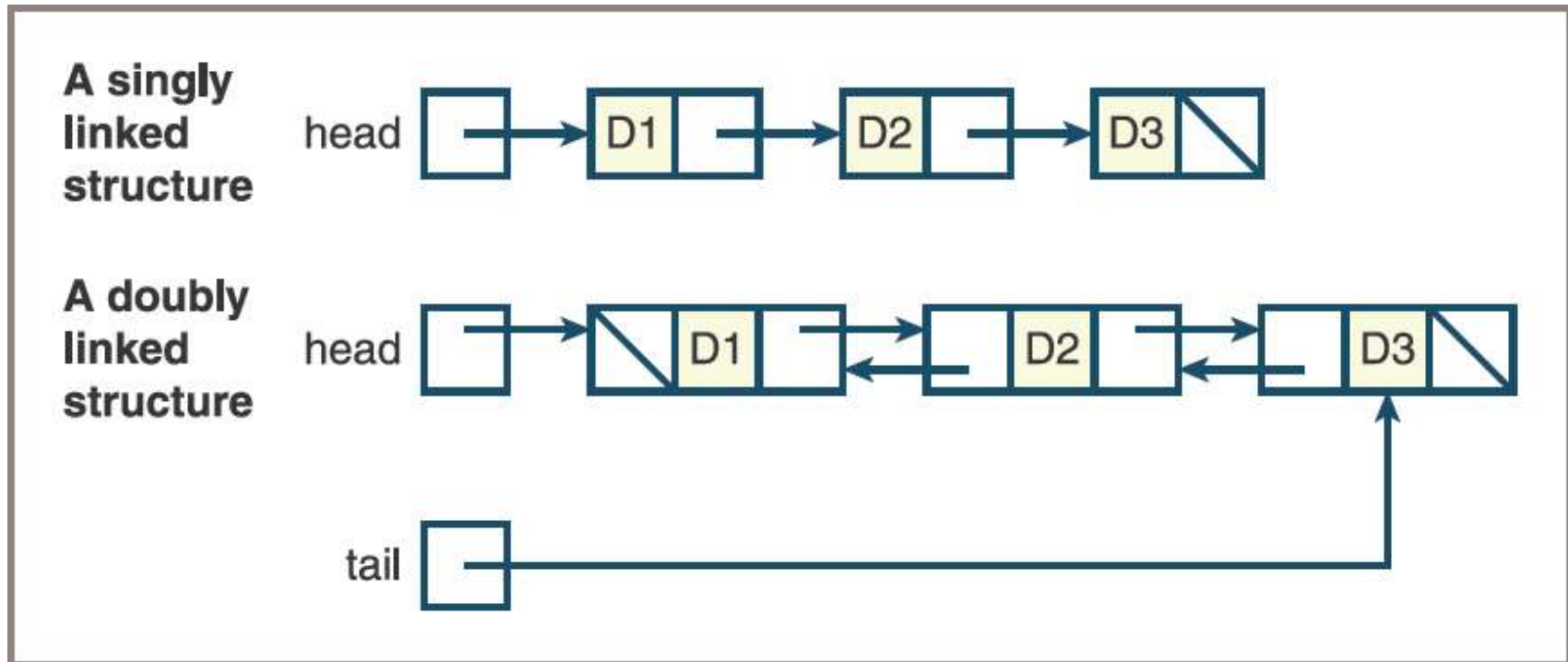
- Linked structure
 - A concrete data type that implements many types of collections
- This section covers characteristics that programmers must keep in mind when using linked structures to implement any type of collection

Singly Linked Structures and Doubly Linked Structures (1 of 4)

- The two simplest linked structures
 - Singly linked structure
 - Doubly linked structure
- It is useful to draw diagrams of linked structures using a box and pointer notation (see Figure 4.6 on next slide)
- A user of a singly linked structure accesses the first item by following a single external head link:
 - User then accesses other items by chaining through the single links that emanate from the items
 - Easy to get the successor of an item
 - Not so easy to get to the predecessor of an item

Singly Linked Structures and Doubly Linked Structures (2 of 4)

Figure 4-6: Two types of linked structures



Singly Linked Structures and Doubly Linked Structures (3 of 4)

- A doubly linked structure contains links running in both directions:
 - A second external link, called the *tail link*, allows the user of a doubly linked structure to access the last item directly
- The last item in either type of linked structure has no link to the next item:
 - Figure indicates the absence of a link, called an *empty link*, by means of a slash instead of an arrow
- A programmer who uses a linked structure cannot immediately access an item by specifying its index position:
 - Programmer must start at one end of the structure and follow the links until the desired position (item) is reached

Singly Linked Structures and Doubly Linked Structures (4 of 4)

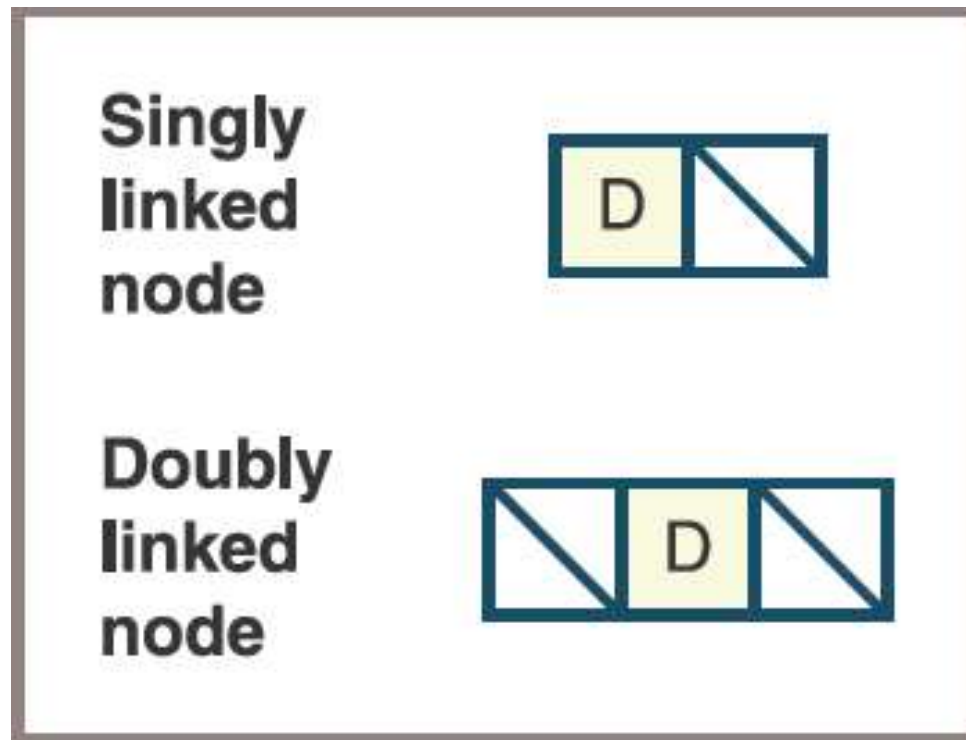
- The way in which memory is allocated for linked structures is unlike that of arrays and has two important consequences for insertion and removal operations:
 - Once an insertion or removal point has been found, the insertion or removal can take place with no shifting of data items in memory
 - The linked structure can be resized during each insertion or removal with no extra memory cost and no copying of data items

Noncontiguous Memory and Nodes (1 of 4)

- A linked structure decouples the logical sequence of items in the structure from any ordering in memory:
 - The cell for a given item in a linked structure can be found anywhere in memory as long as the computer can follow a link to its address or location
 - Called noncontiguous memory
- Basic unit of representation in a linked structure is a *node*
- A *singly linked node* contains the following components or fields:
 - A data item
 - A link to the next node in the structure (a *doubly linked node* contains a link to the previous node in the structure as well)

Noncontiguous Memory and Nodes (2 of 4)

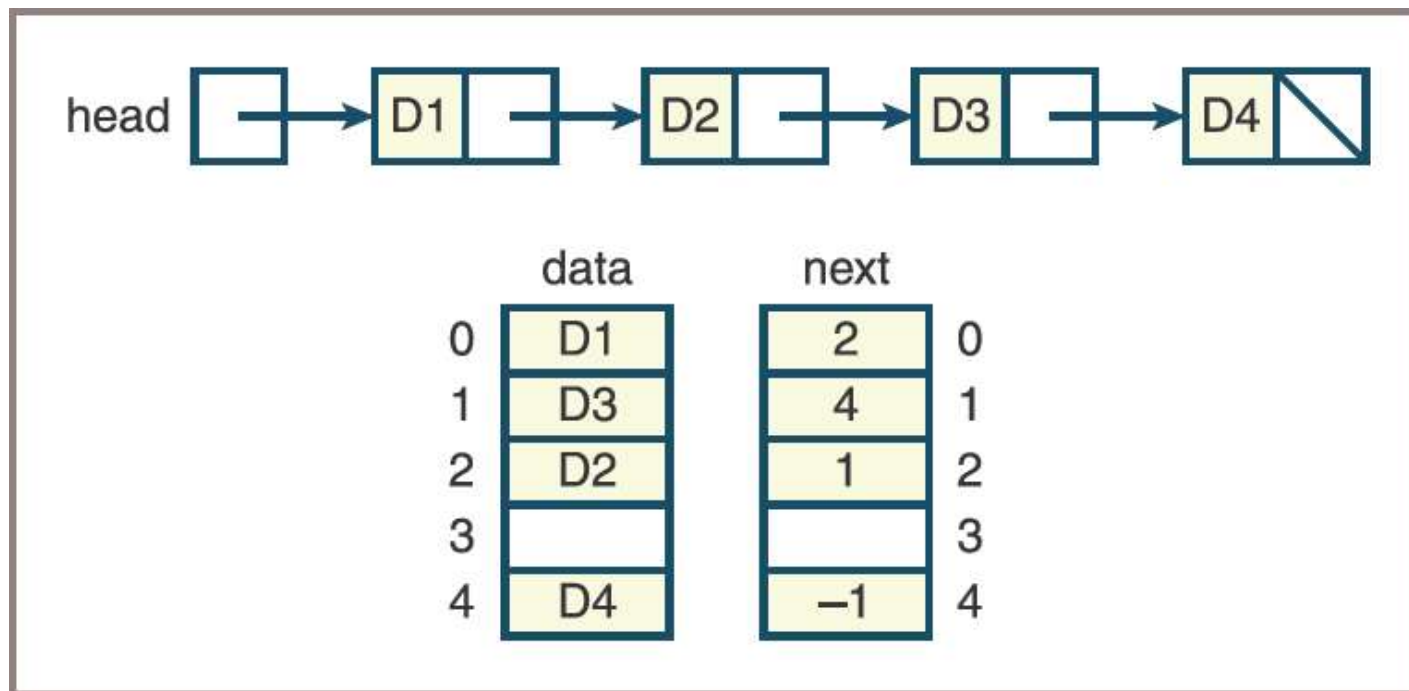
Figure 4-7: Two types of nodes with empty links



Noncontiguous Memory and Nodes (3 of 4)

- Figure 4.8 shows a linked structure and its array representation

Figure 4-8: An array representation of a linked structure



Noncontiguous Memory and Nodes (4 of 4)

- In more modern languages the programmer has direct access to the addresses of data in the form of *pointers*:
 - A node in a singly linked structure contains a data item and a pointer value
- Python programmers set up nodes and linked structures by using *references* to objects:
 - Programmers define a singly linked node by defining an object that contains two fields: a reference to a data item and a reference to another node

Defining a Singly Linked Node Class

- Code for a singly linked node class:

```
class Node(object):  
    """Represents a singly linked node."""  
  
    def __init__(self, data, next = None):  
        """Instantiates a Node with a default next of None."""  
        self.data = data  
        self.next = next
```

Using the Singly Linked Node Class (1 of 4)

- Node variables are initialized to either the **None** value or a new **Node** object
- Code segment that shows some variations on these two options:

```
# Just an empty link
```

```
node1 = None
```

```
# A node containing data and an empty link
```

```
node2 = Node("A", None)
```

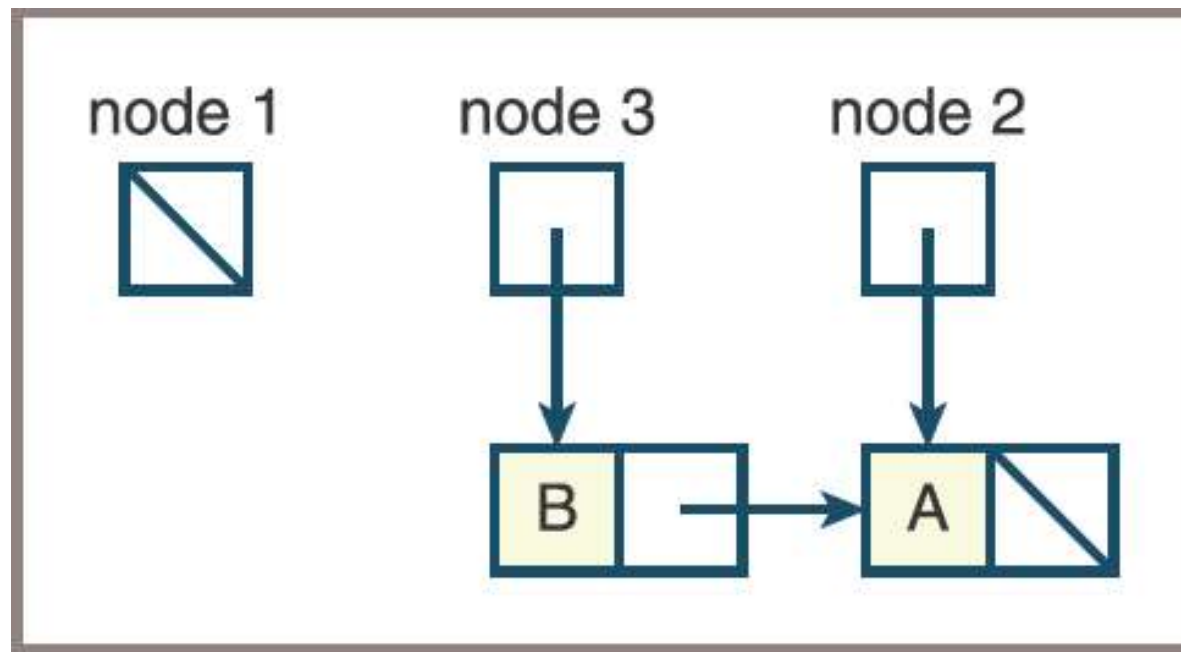
```
# A node containing data and a link to node2
```

```
node3 = Node("B", node2)
```


Using the Singly Linked Node Class (2 of 4)

- Figure 4.9 shows the state of the three variables after this code is run

Figure 4-9: Three external links



Using the Singly Linked Node Class (3 of 4)

- Suppose you attempt to place the first node at the beginning of the linked structure that already contains **node2** and **node3** by running the following statement:

```
node1.next = node3
```

- Python responds by raising an **AttributeError**

- To create the desired link, you could run either

```
node1 = Node("C", node3)
```

or

```
node1 = Node("C", None)
```

```
node1.next = node3
```

Using the Singly Linked Node Class (4 of 4)

- Linked structures are processed with loops:
 - Use loops to create a linked structure and visit each node in it
- Code to create a singly linked structure and print its contents:

```
File: testnode.py
Tests the Node class.
"""
```

```
from node import Node
head = None
# Add five nodes to the beginning of the linked structure
for count in range(1, 6):
    head = Node(count, head)
# Print the contents of the structure
while head != None:
    print(head.data)
    head = head.next
```

Operations on Singly Linked Structures

- The programmer must emulate index-based operations on a linked structure by manipulating links within the structure
- This section covers how these manipulations are performed in common operations, such as
 - Traversals
 - Insertions
 - Removals

Traversal (1 of 2)

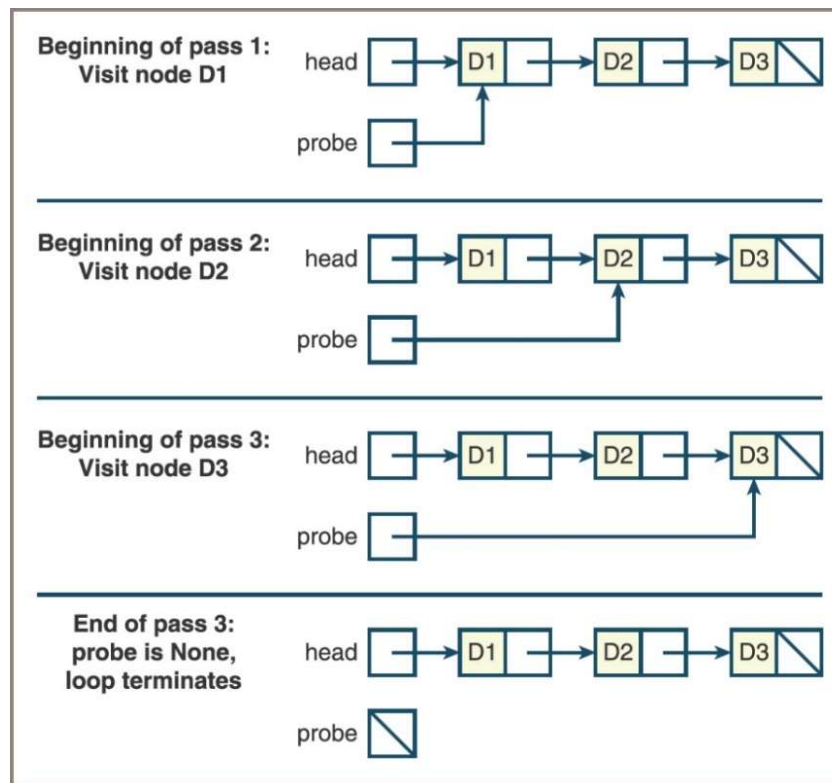
- Traversal
 - Uses a temporary pointer variable named **probe**
 - This variable is initialized to the linked structure's **head** pointer and then controls a loop as follows:

```
probe = head
while probe != None:
    <use or modify probe.data>
    probe = probe.next
```

Traversal (2 of 2)

- Figure 4.10 shows the state of the pointer variables **probe** and **head** during each pass of the loop

Figure 4-10: Traversing a linked structure



Searching (1 of 2)

- The sequential search of a linked structure resembles a traversal:
 - You must start at the first node and follow the links until you reach a sentinel
- Two possible sentinels
 - The empty link, indicating there are no more data items to examine
 - A data item that equals the target item, indicating a successful search
- The form of the search for a given item:

```
probe = head
while probe != None and targetItem != probe.data:
    probe = probe.next
if probe != None:
    <targetItem has been found>
else:
    <targetItem is not in the linked structure>
```

Searching (2 of 2)

- Accessing the i th item of a linked structure is also a sequential search operation
- Here is the form for accessing the i th item:

```
# Assumes 0 <= index < n
probe = head
while index > 0:
    probe = probe.next
    index -= 1
return probe.data
```


Replacement (1 of 2)

- Replacement operations also employ the traversal pattern:
 - Search for a given item or position in the linked structure and replace the item with a new item
- The form of the operation:

```
probe = head
while probe != None and targetItem != probe.data:
    probe = probe.next
if probe != None:
    probe.data = newItem
    return True
else:
    return False
```

Replacement (2 of 2)

- The operation to replace the i th item assumes that $0 \leq i < n$
- Here is the form:

```
# Assumes  $0 \leq \text{index} < n$   
probe = head  
while index > 0:  
    probe = probe.next  
    index -= 1  
probe.data = newItem
```

Inserting at the Beginning (1 of 2)

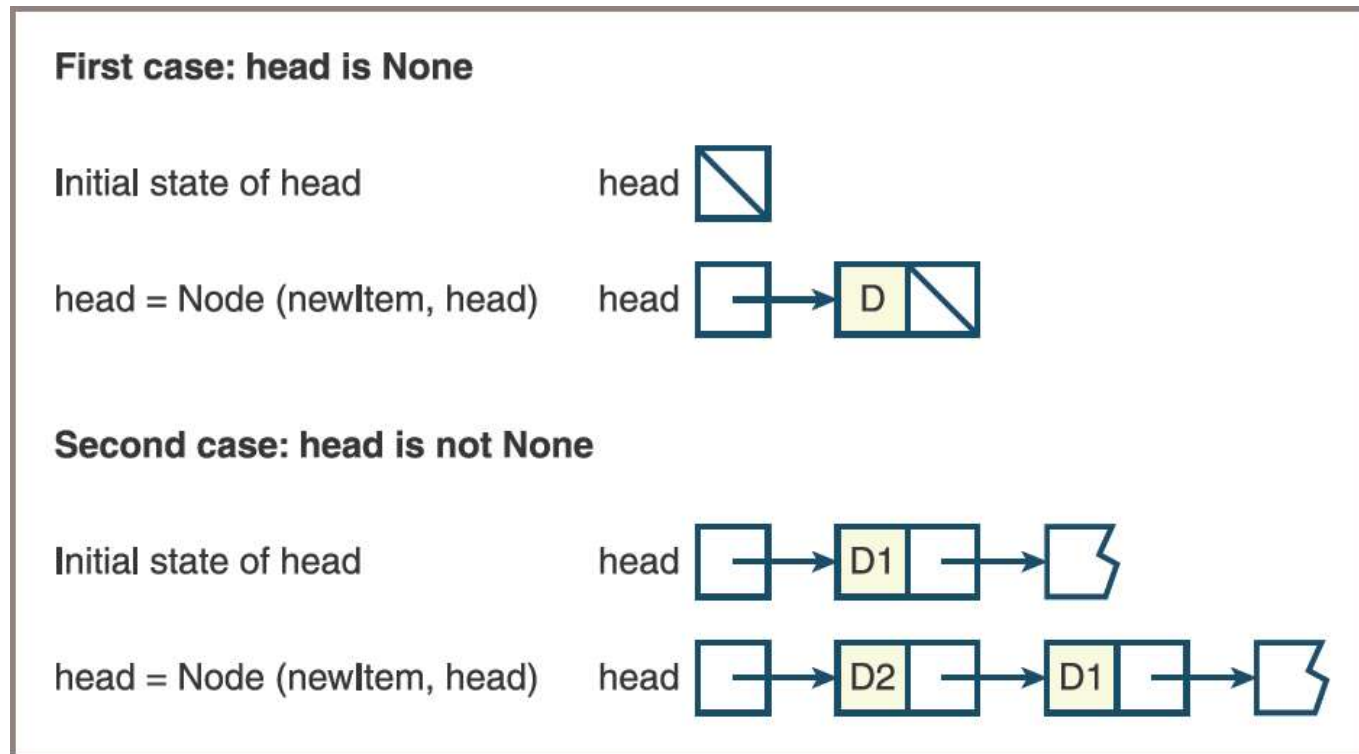
- The form:

```
head = Node (newItem, head)
```

- Figure 4.11 traces this operation for two cases:
 - The pointer is **None** in the first case, so the item is inserted into the structure
 - In the second case, the second item is inserted at the beginning of the same structure

Inserting at the Beginning (2 of 2)

Figure 4-11: The two cases of inserting an item at the beginning of a linked structure



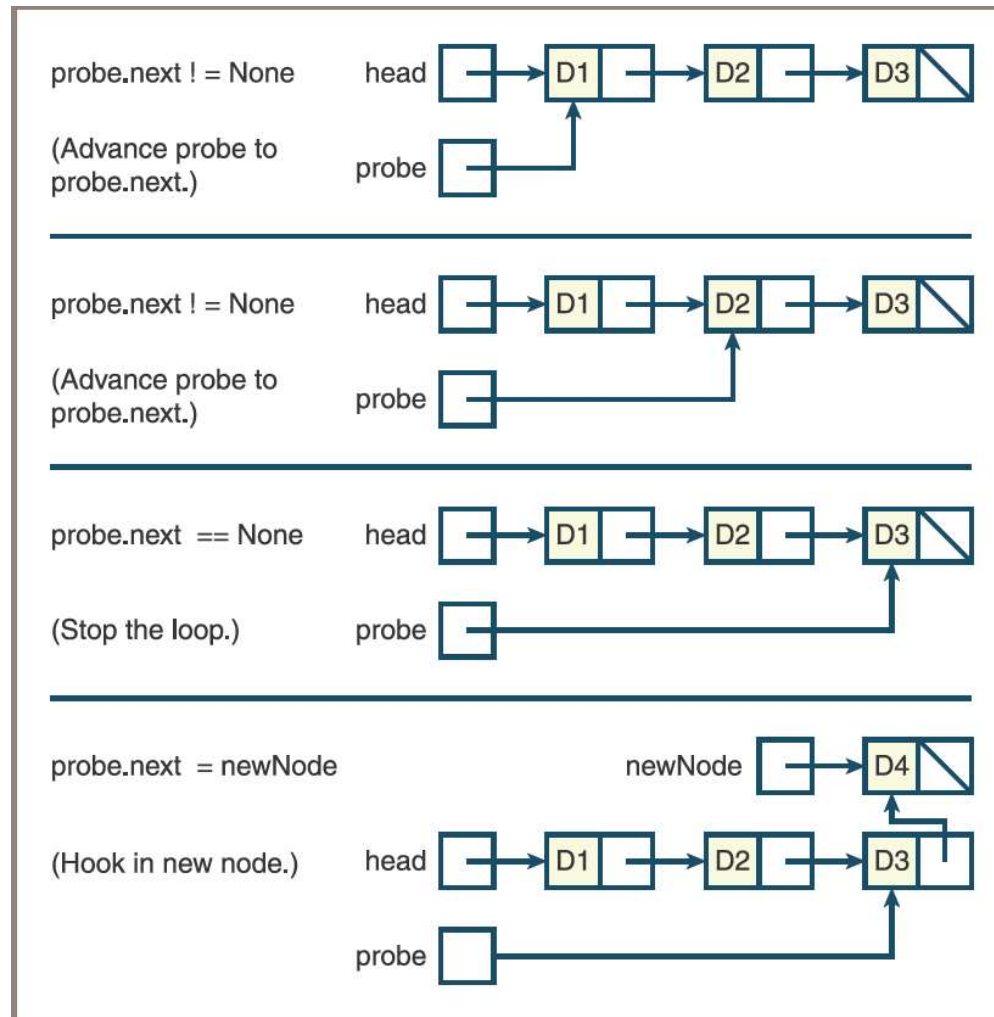
Inserting at the End (1 of 2)

- Inserting an item at the end of a singly linked structure must consider two cases:
 - The **head** pointer is **None**, so the head pointer is set to the new node
 - The **head** pointer is not **None**, so the code searches for the last node and aims its next pointer at the new node
- The second case returns you to the traversal pattern. Here is the form:

```
newNode = Node (newItem)
if head is None:
    head = newNode
else:
    probe = head
    while probe.next != None:
        probe = probe.next
    probe.next = newNode
```

Inserting at the End (2 of 2)

Figure 4-12: Inserting an item at the end of a linked structure



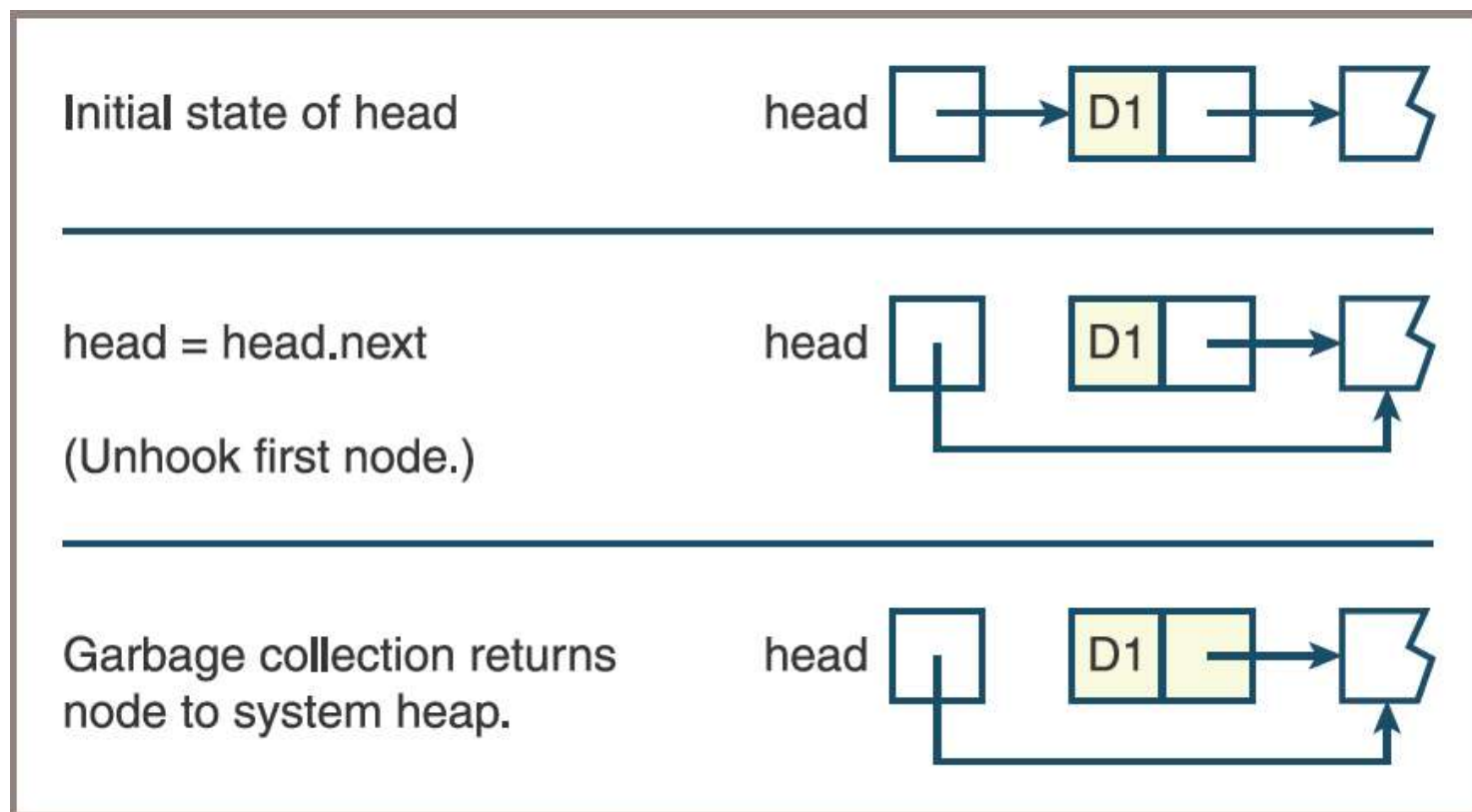
Removing at the Beginning (1 of 2)

- Here is the form:

```
# Assumes at least one node in the structure
removedItem = head.data
head = head.next
return removedItem
```

Removing at the Beginning (2 of 2)

Figure 4-13: Removing an item at the beginning of a linked structure



Removing at the End (1 of 3)

- Removing an item at the end of a singly linked structure assumes at least one node in the structure
- Two cases to consider
 - There is just one node and the **head** pointer is set to **None**
 - There is a node before the last node and the code searches for this second-to-last node and sets its next pointer to **None**
- In either case, the code returns the data item contained in the deleted node

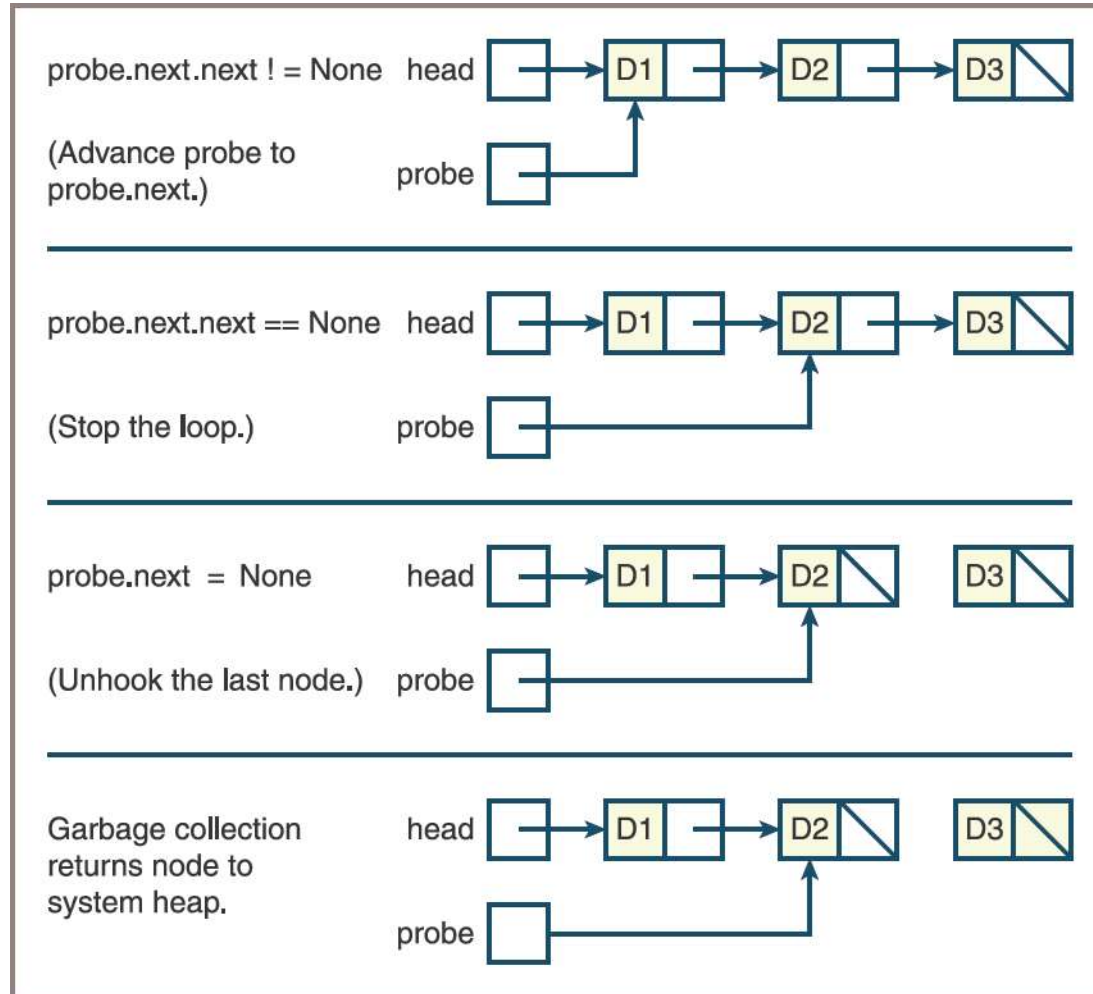
Removing at the End (2 of 3)

- Here is the form:

```
# Assumes at least one node in structure
removedItem = head.data
if head.next is None:
    head = None
else:
    probe = head
    while probe.next.next != None:
        probe = probe.next
    removedItem = probe.next.data
    probe.next = None
return removedItem
```

Removing at the End (3 of 3)

Figure 4-14: Removing an item at the end of a linked structure



Inserting at Any Position (1 of 3)

- The case of an insertion at the beginning uses the code presented earlier
- In the case of an insertion at some other position i ,
 - The operation must first find the node at position $i - 1$ (if $i < n$) or the node at position $n - 1$ (if $i \geq n$)
- There are two cases to consider:
 - That node's next pointer is **None**
 - That node's next pointer is not **None**

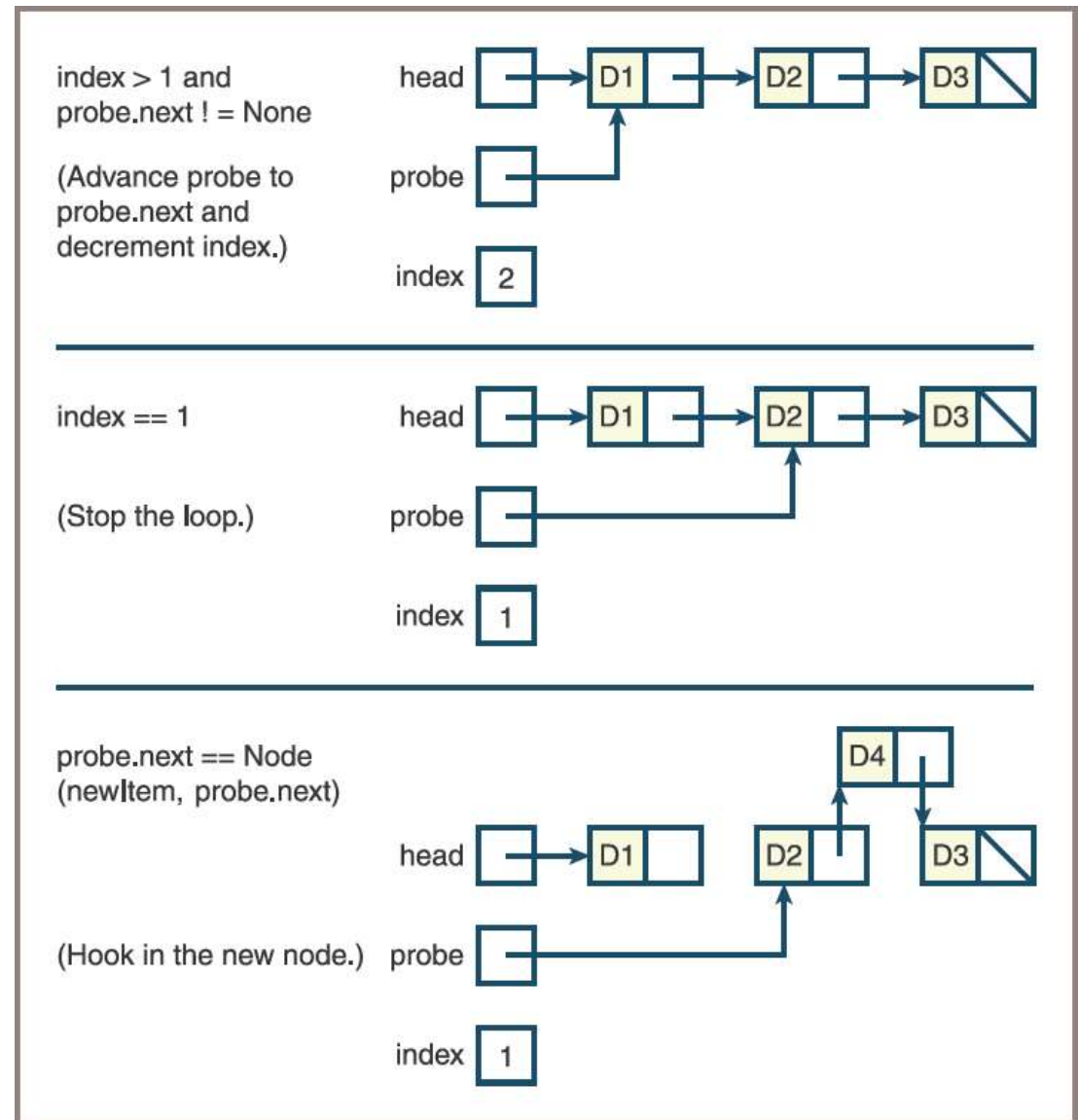
Inserting at Any Position (2 of 3)

- Here is the form:

```
if head is None or index <= 0:
    head = Node(newItem, head)
else:
    # Search for node at position index - 1 or the last
    # position
    probe = head
    while index > 1 and probe.next != None:
        probe = probe.next
        index -= 1
    # Insert new node after node at position index - 1
    # or last position
    probe.next = Node(newItem, probe.next)
```

Inserting at Any Position (3 of 3)

Figure 4-15: Inserting an item between two items in a linked structure



Removing at Any Position (1 of 3)

- The removal of the i th item from a linked structure has three cases:
 - $i \leq 0$ —You use the code to remove the first item
 - $0 < i < n$ —You search for the node at position $i - 1$, as in insertion, and remove the following node
 - $i \geq n$ —You remove the last node

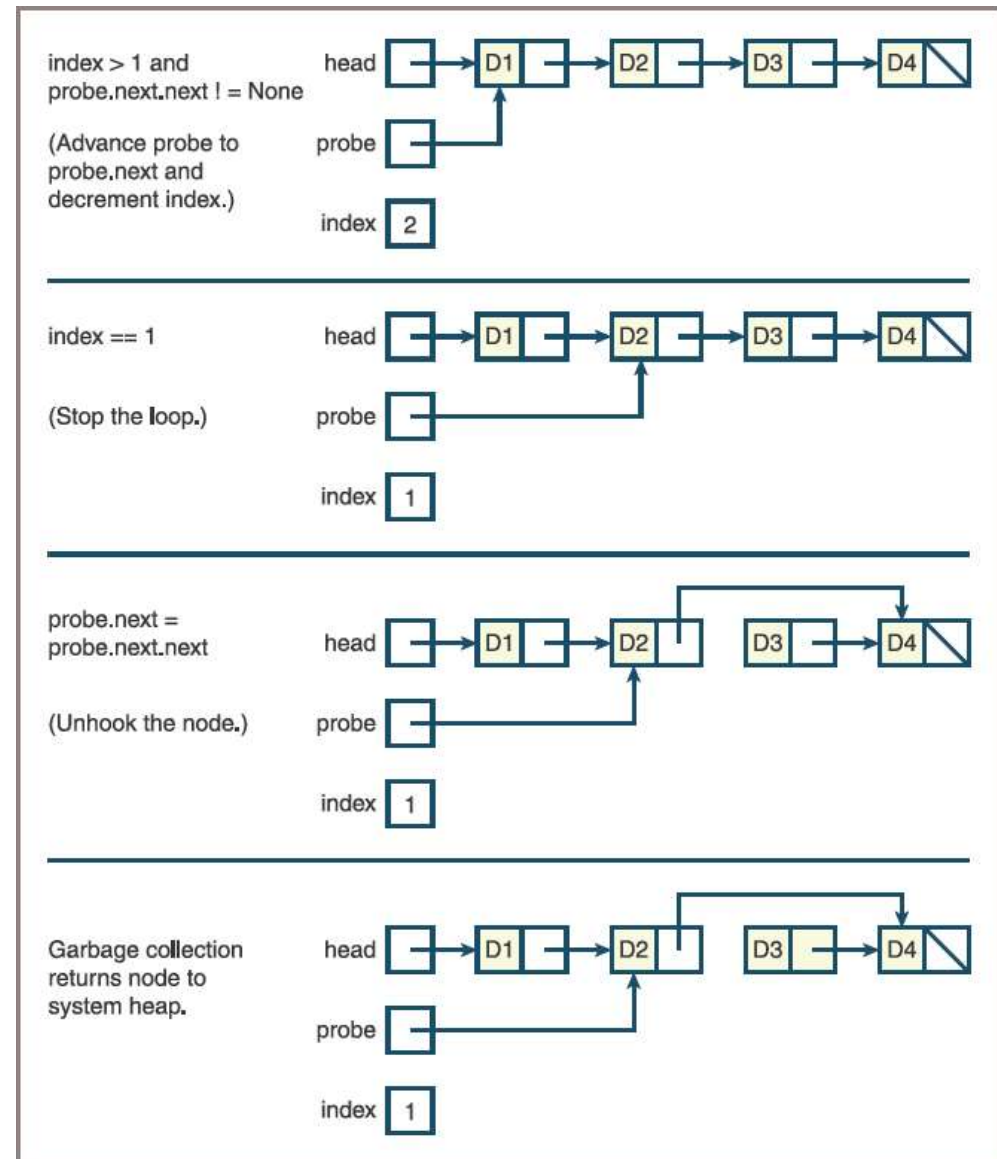
Removing at Any Position (2 of 3)

- Here is the form:

```
# Assumes that the linked structure has at least one item
if index <= 0 or head.next is None:
    removedItem = head.data
    head = head.next
    return removedItem
else:
    # Search for node at position index - 1 or
    # the next to last position
    probe = head
    while index > 1 and probe.next.next != None:
        probe = probe.next
        index -= 1
    removedItem = probe.next.data
    probe.next = probe.next.next
    return removedItem
```


Removing at Any Position (3 of 3)

Figure 4-16: Removing an item between two items in a linked structure



Complexity Trade-Off: Time, Space, and Singly Linked Structures

- Table 4.3 The Running Times of Operations on Singly Linked Structures

Operation	Running Time
Access at i th position	$O(n)$, average case
Replacement at i th position	$O(n)$, average case
Insert at beginning	$O(1)$, best and worst case
Remove from beginning	$O(1)$, best and worst case
Insert at i th position	$O(n)$, average case
Remove from i th position	$O(n)$, average case

Variations on a Link

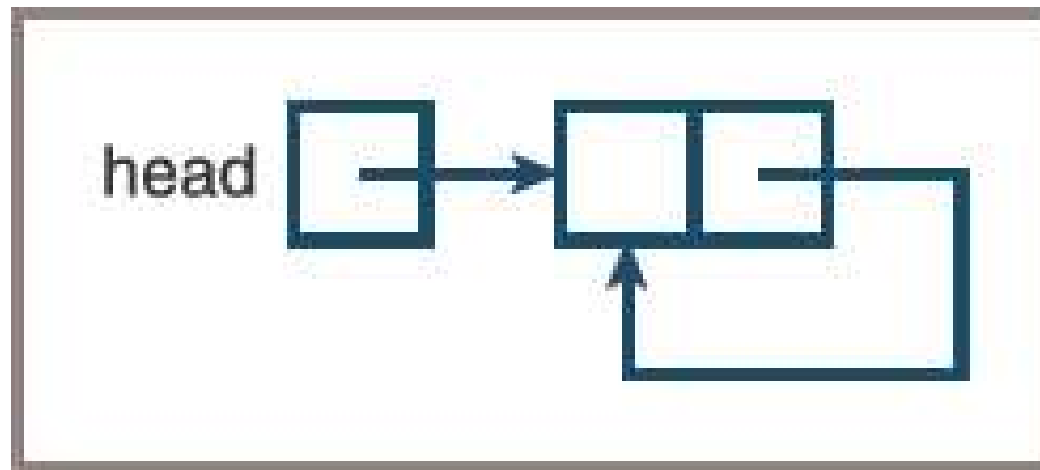
- This section examines two types of linked structures with extra pointers that can help to improve performance and simplify code

A Circular Linked Structure with a Dummy Header Node (1 of 4)

- The insertion and the removal of the first node are special cases of the insert i th and remove i th operations on singly linked structures:
 - These cases are special because the **head** pointer must be reset
- You can simplify these operations by using a *circular linked structure with a dummy header node*:
 - A circular linked structure contains a link from the last node back to the first node in the structure
 - This node, the dummy header node, contains no data but serves as a marker for the beginning and the end of the linked structure

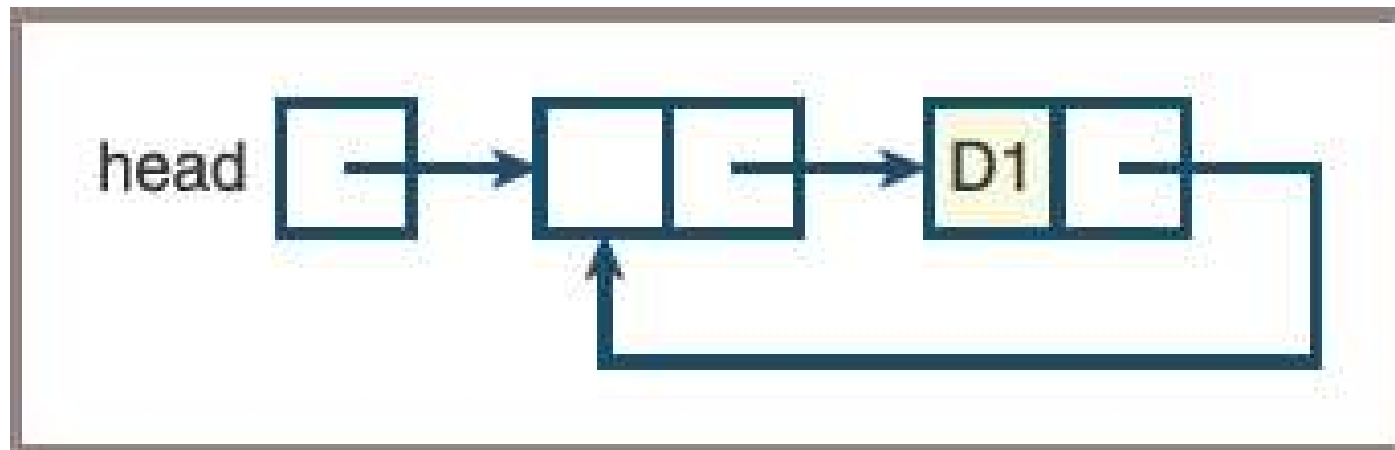
A Circular Linked Structure with a Dummy Header Node (2 of 4)

Figure 4-17: An empty circular linked structure with a dummy header node



A Circular Linked Structure with a Dummy Header Node (3 of 4)

Figure 4-18: A circular linked structure after inserting the first node



A Circular Linked Structure with a Dummy Header Node (4 of 4)

- Code for insertions at the i th position using the new representation of a linked structure:

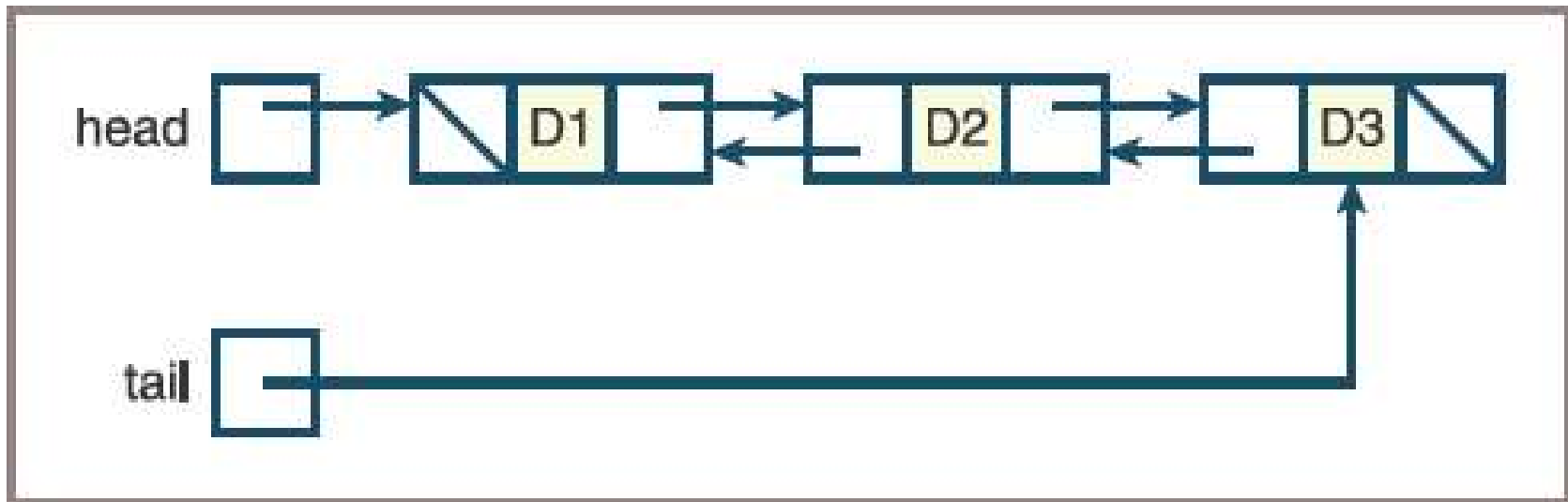
```
# Search for node at position index - 1 or the last position
probe = head
while index > 0 and probe.next != head:
    probe = probe.next
    index -= 1
# Insert new node after node at position index - 1 or
# last position
probe.next = Node(newItem, probe.next)
```

Doubly Linked Structures (1 of 4)

- A doubly linked structure has the advantages of a singly linked structure
- In addition, it allows the user to do the following:
 - Move left, to the previous node, from a given node
 - Move immediately to the last node
- Figure 4.19 shows a doubly linked structure that contains three nodes:
 - Note the presence of two pointers, conventionally known as **next** and **previous**, in each node
 - Note also the presence of a second external **tail** pointer that allows direct access to the last node in the structure

Doubly Linked Structures (2 of 4)

Figure 4-19: A doubly linked structure with three nodes



Doubly Linked Structures (3 of 4)

- Code:

```
class Node(object):
```

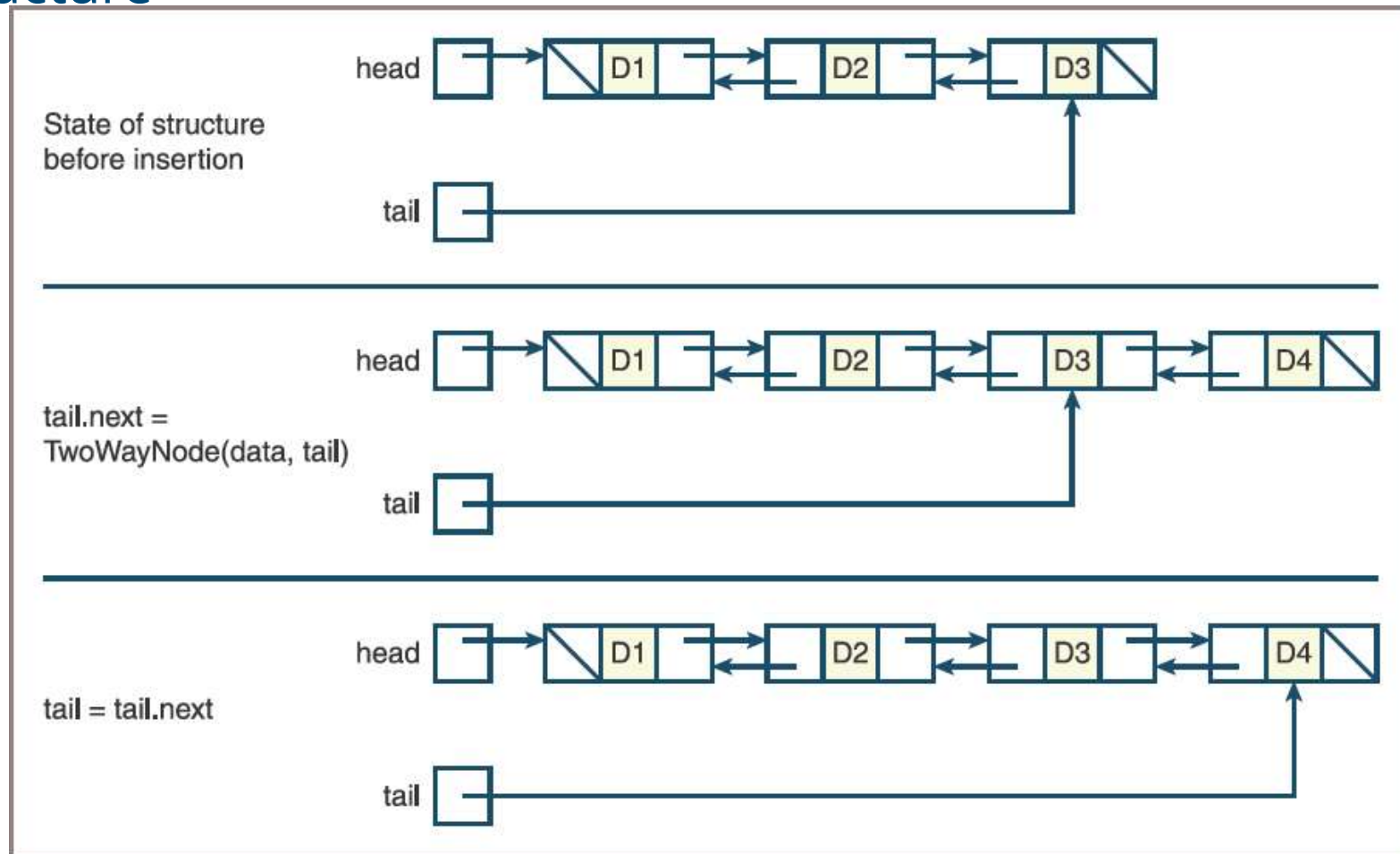
```
    def __init__(self, data, next = None):  
        """Instantiates a Node with default next of None"""  
        self.data = data  
        self.next = next
```

```
class TwoWayNode(Node):
```

```
    def __init__(self, data, previous = None, next = None):  
        """Instantiates a TwoWayNode."""  
        Node.__init__(self, data, next)  
        self.previous = previous
```

Doubly Linked Structures (4 of 4)

Figure 4-20: Inserting an item at the end of a doubly linked structure



Chapter Summary (1 of 2)

- A data structure is an object that represents the data contained in a collection
- The array is a data structure that supports random access to an item by position
- A two-dimensional array locates each data value at a row and column in a rectangular grid
- A linked structure is a data structure that consists of zero or more nodes
- A singly linked structure's nodes contain a data item and a link to the next node

Chapter Summary (2 of 2)

- Insertions or removals in linked structures require no shifting of data elements
- Using a header node in a linked structure can simplify some of the operations
 - Such as adding or removing items